NIRMA UNIVERSITY

Institute of Technology

Department of Computer Science and Engineering

INNOVATIVE ASSIGNMENT REPORT

PacketQuest

Stop-and-Wait ARQ Protocol Simulation

Reliable data delivery over a noisy channel, explained with an Amazon order example

Detail	Information
Subject	Computer Networks
Topic	Data Link Layer: Error Control (Stop-and-Wait ARQ with 1-bit sequence numbers)
Roll No. / Name	25BCE533 (Om Vaghela)
Roll No. / Name	25BCE535 (Raj Vyas)
Faculty	Dr. Jigna Hathaliya
Academic Year	2026-27

1. Introduction

When data travels from one computer to another, it passes through a channel that is never perfect. Noise, interference and congestion can corrupt or completely drop a frame on the way. If the sender never finds out about this, the receiver ends up with missing or broken data. To avoid this, the data link layer uses error control, and one of its simplest and most important techniques is Automatic Repeat Request (ARQ).

Stop-and-Wait ARQ is the most basic ARQ protocol. The sender sends exactly one frame, then stops and waits for an acknowledgement (ACK) from the receiver. Only when the ACK arrives does the sender move on to the next frame. If the ACK does not arrive within a fixed timeout, the sender assumes something went wrong and sends the same frame again.

This report describes PacketQuest, an interactive web-based simulation of Stop-and-Wait ARQ. To make the idea easy to relate to, the simulation uses an everyday scenario: a customer placing an order on Amazon. The customer is the sender, the Amazon server is the receiver, and the order confirmation is the ACK. Users can switch the channel between noiseless and noisy, change the packet-loss probability, and watch every transmission, loss, timeout and retransmission happen step by step.

2. Objectives

- To simulate the Stop-and-Wait ARQ protocol and show how it works on both an ideal (noiseless) and an error-prone (noisy) channel.
- To demonstrate the role of acknowledgements, timeouts and retransmissions in reliable data transfer.
- To show how a 1-bit sequence number (0 or 1) lets the receiver tell a new frame from a duplicate.
- To explain a networking concept through a real-world analogy (an online order) so that it is easy to remember.
- To give live statistics (successful deliveries, lost frames, retransmissions, duplicates, average delivery time) so the cost of noise can be seen in numbers.

3. Real-World Analogy: The Amazon Order

Every part of the protocol is mapped to something that happens when we shop online. This mapping is the main innovative idea of the project and is used on the theory page and in the simulation itself.

Table 1: Mapping of protocol terms to the Amazon order example

Networking term	Amazon example	Meaning in the simulation
Sender	Customer	Places one order (frame) at a time and waits for confirmation.
Receiver	Amazon server	Receives the order, checks the sequence number, and replies with a confirmation.
Data frame	Order request	Shown as a box labelled with the order number and sequence number (for example ORD#1 SEQ:0).
ACK	Order confirmation	Green box travelling back from server to customer, carrying the sequence number.
Noisy channel	Bad internet connection	Randomly drops frames or ACKs based on the loss probability.
Timeout	No confirmation email arrives	Timer bar of 2 seconds; when it ends, the same order is placed again.
Duplicate frame	Same order sent twice	Detected by comparing sequence numbers; the extra copy is discarded.

4. Stop-and-Wait ARQ Protocol

In Stop-and-Wait ARQ, the sender keeps only one frame in transit at any moment. Each frame carries a sequence number which alternates between 0 and 1. Because only one frame is outstanding, a single bit is enough: the receiver always knows whether it expects a 0 or a 1 next.

4.1 Key concepts

- **Acknowledgement (ACK):** A small control message from the receiver which tells the sender that a frame arrived correctly.
- **Timeout:** The time the sender is willing to wait for an ACK. In PacketQuest the timer runs for 2 seconds and is shown as a progress bar.
- **Retransmission:** If the timer expires, the sender sends the same frame again with the same sequence number.
- **1-bit sequence number:** Alternates 0, 1, 0, 1 for each new frame. A retransmission keeps the old number, a new frame uses the toggled number.
- **Duplicate detection:** The receiver keeps the sequence number it expects next. A frame with a different number is a duplicate, so it is discarded, but an ACK is still sent so the sender can move on.

4.2 Working steps

1. **Send:** The sender transmits one frame with sequence number 0 or 1 and starts the timer.
2. **Wait:** The sender stops and waits for the ACK of that frame.
3. **Receive:** If the frame reaches the receiver, it checks the sequence number and sends back an ACK carrying the same number.
4. **Success:** When the ACK reaches the sender before the timer ends, the sender toggles its sequence number and sends the next frame.
5. **Timeout:** If no ACK arrives in time (the frame or the ACK was lost), the sender retransmits the same frame with the same sequence number.
6. **Repeat:** Steps 1 to 5 repeat for every new frame.

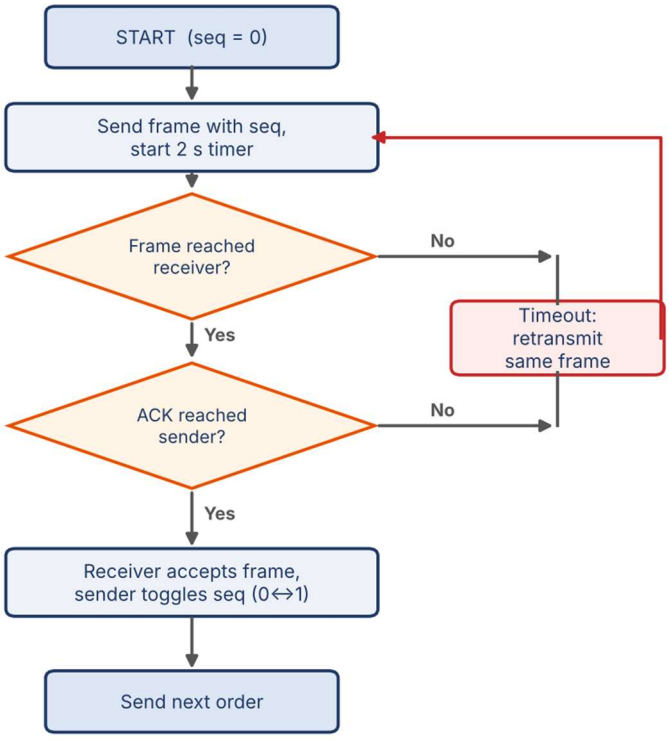


Figure 1: Flowchart of the Stop-and-Wait ARQ logic used in PacketQuest

5. Transmission Scenarios

The protocol has three possible outcomes for any frame. The timeline diagrams below show each case, with the sender on the left and the receiver on the right. Time moves downwards.

5.1 Scenario 1: Normal delivery (no loss)

The frame reaches the receiver, the ACK comes back in time, and the sender toggles the sequence number to 1 for the next order. This is what always happens when the noisy channel is switched off.

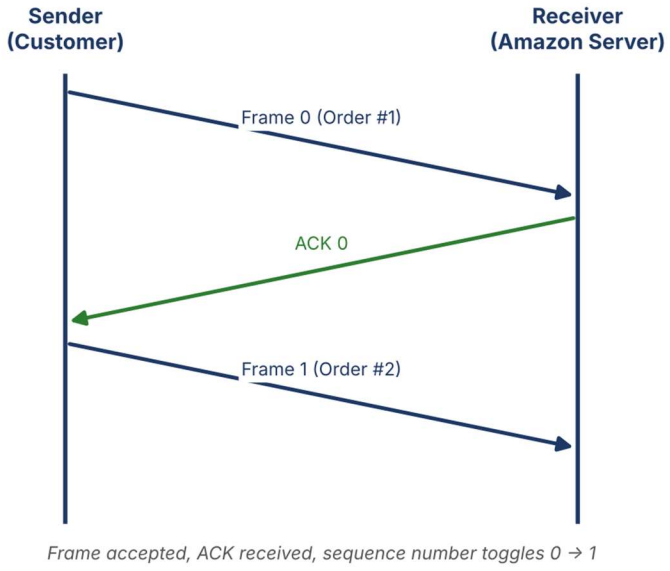


Figure 2: Normal delivery on a noiseless channel

5.2 Scenario 2: Data frame lost

The frame disappears in the channel, so the receiver never sees it and sends nothing. The sender's 2-second timer runs out and it retransmits Frame 0. The second copy gets through and is acknowledged.

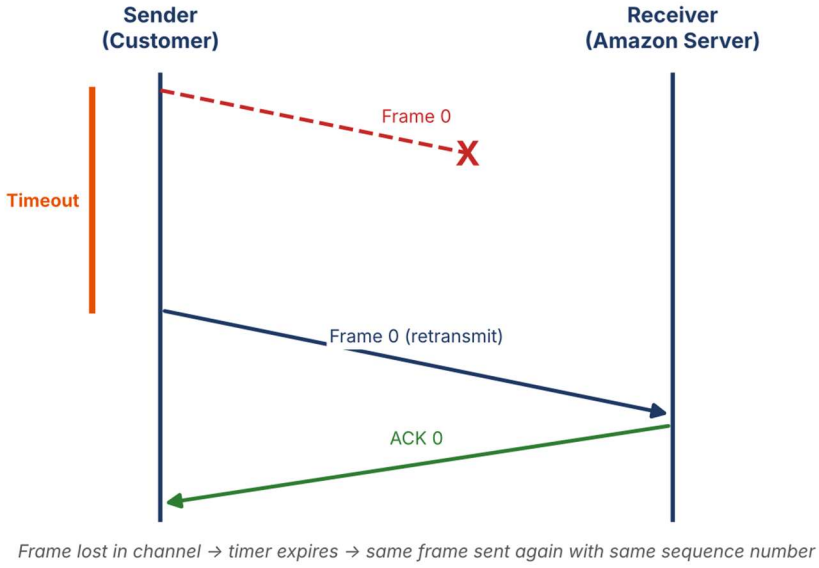


Figure 3: Data frame lost, timeout, then retransmission

5.3 Scenario 3: ACK lost

This case is more interesting. The receiver did get the frame and sent an ACK, but the ACK was lost on the way back. The sender cannot tell the difference between a lost frame and a lost ACK, so it times out and retransmits. In a full implementation, the receiver would now see Frame 0 for the second time and use the sequence number to recognise it as a duplicate. It discards the copy but sends the ACK again. This is exactly why sequence numbers are needed.

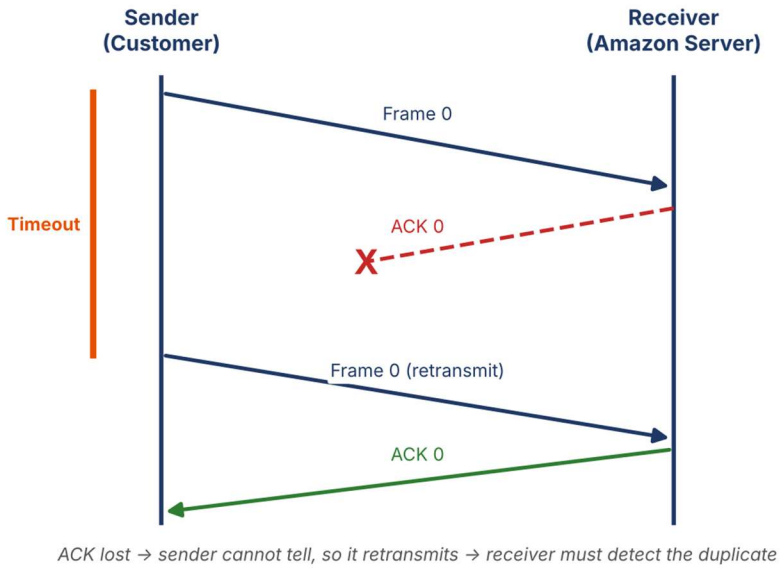


Figure 4: ACK lost, timeout, retransmission and duplicate handling

6. System Design and Implementation

PacketQuest is a single-page web application. It runs completely in the browser and needs no server or installation, so it can be opened on any computer by double-clicking the HTML file.

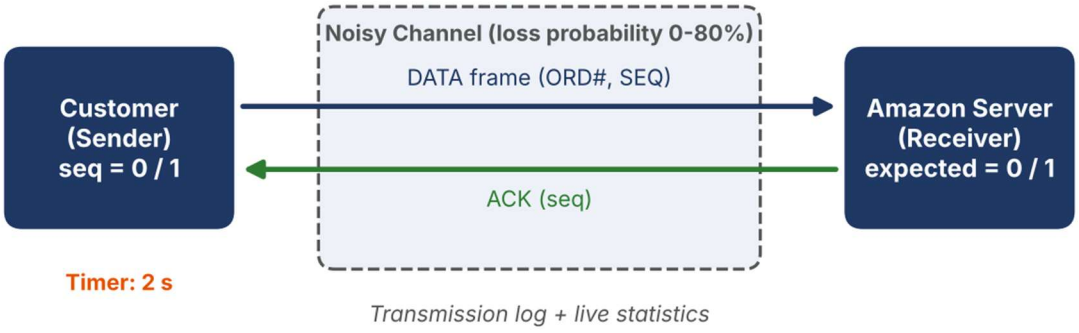


Figure 5: Block diagram of the simulated system

6.1 Technologies used

Table 2: Technologies used

Technology	Purpose
HTML5	Structure of the landing page, theory page and simulation page.
CSS and Tailwind CSS	Glass-style cards, gradient background, animations (packet movement, pulse, fade-out, timeout bar) and dark / light mode.
JavaScript	All simulation logic: packet animation, random loss, timeout, sequence number toggling, statistics and logging.
Google Font (Inter)	Clean and readable text throughout the interface.

6.2 State variables

The whole simulation is controlled by a small set of variables kept in one state object.

Table 3: Main state variables

Variable	Purpose
senderSeqNum	Sequence number (0 or 1) the sender is currently using.
receiverExpectedSeq	Sequence number (0 or 1) the receiver expects next.
orderNumber	Running order number, used only for display (ORD#1, ORD#2, ...).
isNoisy	True when the noisy channel switch is on.
lossProbability	Chance (in percent) that a frame or ACK is lost; set by the slider from 0 to 80, default 30.
stats	Counters for successes, lost frames, retransmissions, duplicates and total delivery time.
isTransmitting	Prevents a new order from being sent while the previous one is still in progress.

6.3 Main functions

Table 4: Main functions

Function	What it does
animatePacket()	Core of the protocol. Sends the frame, checks for loss, runs the receiver check, sends the ACK, checks for ACK loss, and retransmits through a recursive call after a timeout.
shouldPacketFail()	Returns true if the channel is noisy and a random number is below the loss probability.
showTimeoutBar()	Animates the 2-second timeout timer bar.
addLog()	Adds a colour-coded, time-stamped line to the transmission log.
updateStats()	Refreshes the five statistics shown in the control panel.
updateSequenceDisplay()	Shows the current sender and receiver sequence numbers.
Reset handler	Clears all counters, sets both sequence numbers back to 0 and empties the log.

6.4 Simulation timing

The animation uses fixed delays so that each stage can be followed by the eye. These values also decide the delivery times shown in the statistics.

Table 5: Approximate timing in the simulation

Stage	Time
Frame travelling to the receiver	about 1.1 s
Receiver checks the sequence number and prepares the ACK	about 1.1 s
ACK travelling back to the sender	about 1.1 s
Timeout period after a loss	2.0 s
Normal delivery with no loss (total)	about 3.3 s
Extra time added by each lost frame / lost ACK	about 3.6 s / 4.2 s

7. User Interface and Features

7.1 Pages

- **Landing page:** Title, a short description, and two buttons: Learn Theory and Start Simulation.

- **Theory page:** Cards that explain Sender, Receiver, ACK and Noisy Channel, the working steps, and key concepts such as timeout and 1-bit sequence numbers.
- **Simulation page:** A control panel on the left, the animated channel in the middle, a status message, a scrollable transmission log, and a conclusion section with advantages and limitations.

7.2 Control panel

Table 6: Controls available to the user

Control	Function
Noisy Channel switch	Turns random loss on or off. Off means a perfect, noiseless channel.
Packet Loss Probability slider	Sets the loss chance from 0% to 80% (default 30%).
Send Order button	Starts one transmission. It is disabled until the current order is confirmed.
Reset Simulation button	Resets counters, sequence numbers (back to 0) and the log.
Dark / light mode toggle	Switches the colour theme of the whole application.

7.3 Live statistics

Table 7: Statistics shown during the simulation

Statistic	Meaning
Successful Deliveries	Orders that were accepted and confirmed.
Lost Frames	Frames or ACKs that were dropped by the channel.
Retransmissions	Number of times a frame was sent again after a timeout.
Duplicates Detected	Duplicate frames recognised by the receiver and discarded.
Avg Delivery Time	Total delivery time divided by successful deliveries, in seconds.

8. Noiseless vs Noisy Channel

Table 8: Comparison of channel modes

Aspect	Noiseless channel	Noisy channel
Loss of frames / ACKs	Never happens	Happens randomly with the chosen probability
Retransmissions	None	One or more per order, depending on luck
Timeouts	Never triggered	Triggered after every loss
Delivery time	Constant (about 3.3 s)	Varies; grows as loss probability rises
Role of sequence numbers	Toggle after every order	Also used to spot duplicates after a lost ACK
Real-world example	Order goes through on a fast, stable connection	Order button pressed on weak mobile data

9. Performance Analysis

9.1 Effect of the loss probability

In the simulation both the frame and its ACK can be lost, each with probability p. An attempt therefore succeeds only when both get through, which has probability $(1 - p)^2$. Since the sender keeps trying until one attempt succeeds, the average number of transmissions needed per order is $1 / (1 - p)^2$.

Table 9: Expected transmissions for different loss probabilities

Loss probability p	Success chance per attempt	Average transmissions per order
0%	100%	1.00
10%	81%	1.23
30% (default)	49%	2.04
50%	25%	4.00
80% (maximum)	4%	25.00

The table shows that delivery cost does not grow gently: going from 30% to 50% loss doubles the average work, and at 80% loss an order needs about 25 attempts. This is the reason the slider in the simulation is capped at 80%, and it is easy to see in the retransmission counter.

9.2 Channel utilisation

The sender is busy only while it is putting the frame on the line, and then sits idle until the ACK returns. If Tf is the time to transmit one frame and Tp is the one-way propagation delay, the utilisation of the link is:

$$U = Tf / (Tf + 2 \times Tp) = 1 / (1 + 2a), \quad \text{where } a = Tp / Tf$$

Table 10: Channel utilisation for different values of a (no errors)

a = Tp / Tf	Utilisation U	Comment
0.1	83.3%	Short link, works well
1	33.3%	Link is idle most of the time
10	4.8%	Long-distance link, very wasteful

When the link is long compared to the frame time, almost all the time is spent waiting. This is the main weakness of Stop-and-Wait and the reason for sliding-window protocols.

10. Advantages and Limitations

Advantages

- Very simple to understand and to implement.
- Guarantees reliable delivery, because frames are retransmitted until an ACK arrives.
- Needs only 1 bit for the sequence number and buffers only one frame.
- Duplicates are handled easily using the alternating sequence number.

Limitations

- Low throughput: only one frame is in transit at a time.
- The sender is idle while waiting, so bandwidth is wasted, especially on long or fast links.
- A single loss costs a whole timeout period.
- A badly chosen timeout hurts: too short causes needless duplicates, too long slows recovery.

Limitations of this simulation

- Loss is the only error modelled. Corrupted frames (bit errors detected by a checksum) are not simulated.
- The timeout is fixed at 2 seconds and does not adapt to the channel.

- For simplicity, the receiver's expected sequence number is updated together with the sender's once an order is confirmed. The duplicate-detection branch is written into the code, but in this simplified flow a retransmission after a lost ACK is treated by the receiver like a fresh copy of the same order.
- Only one sender-receiver pair is simulated, and animation delays are fixed, so times are illustrative, not real network measurements.

11. Applications and Future Scope

Where the idea is used

- Simple lock-step protocols such as TFTP send one block and wait for its acknowledgement before the next.
- Small embedded and low-speed serial links, where simplicity matters more than speed.
- Teaching: it is the starting point for understanding Go-Back-N, Selective Repeat and TCP reliability.

Possible improvements

- Add Go-Back-N and Selective Repeat modes so that users can compare throughput.
- Add a corrupted-frame option with checksum verification.
- Let the user change the timeout value and the propagation delay.
- Show a live graph of efficiency versus loss probability.
- Let the user send a batch of orders automatically and export the log.

12. Conclusion

PacketQuest successfully demonstrates Stop-and-Wait ARQ in both noiseless and noisy conditions. By tying each protocol element to a step of an Amazon order, the project makes an abstract data-link-layer concept easy to picture. Animated packets, a timeout bar, a colour-coded log and live statistics let the user see exactly what happens when a frame or ACK is lost, why the sender retransmits, and why sequence numbers are required.

The analysis also shows the price of this simplicity: with one frame in flight, throughput drops quickly as the loss probability or the link length grows. This motivates the more efficient sliding-window protocols. As a learning tool, the simulation is useful for networking students, and it can be extended with additional ARQ protocols in the future.

References

[1] Behrouz A. Forouzan, Data Communications and Networking, McGraw-Hill Education.

[2] Andrew S. Tanenbaum and David J. Wetherall, Computer Networks, Pearson.

[3] James F. Kurose and Keith W. Ross, Computer Networking: A Top-Down Approach, Pearson.

[4] PacketQuest source code (packet_quest_arq.html), project submission.